MiniZinc Sheet

Constraint Programming — quick reference for the lab sessions

1. Getting started

MiniZinc is a modelling language: you describe *what* the solution must satisfy, and a solver (Gecode, Chuffed, OR-Tools CP-SAT…) finds it. You never write the search algorithm yourself.

1. Install the **MiniZinc IDE** (bundled with solvers) from minizinc.org.
2. Write a model in a `.mzn` file; put input values in an optional `.dzn` data file.
3. Pick a solver in the toolbar and press **Run** (Ctrl+R).
4. From the terminal: `minizinc --solver gecode model.mzn data.dzn`

Structure of every model (the order of items is free, but this order is the clearest):

```
% 1. Includes (global constraints)
include "globals.mzn";

% 2. Parameters (known data)
int: n = 4;

% 3. Decision variables (unknowns)
var 1..n: x;
var 1..n: y;

% 4. Constraints
constraint x + y = 5;
constraint x < y;

% 5. Solve item (exactly one)
solve satisfy;

% 6. Output (optional)
output ["x = \(x), y = \(y)\n"];
```

Every item ends with `;` and comments start with `%`.

2. Parameters, variables and types

A **parameter** has a value known before solving; a **decision variable** (`var`) is what the solver must find.

What	Syntax	Example
Integer parameter	`int: name = value;`	`int: n = 8;`
Parameter given later in .dzn	`int: name;`	`int: capacity;`
Integer variable with domain	`var lo..hi: name;`	`var 0..9: S;`
Variable with a set domain	`var {v1, v2, …}: name;`	`var {1, 3, 5}: odd;`
Boolean variable	`var bool: name;`	`var bool: chosen;`
Float variable	`var float: name;`	`var 0.0..10.0: w;`
Range (set of ints)	`set of int: NAME = lo..hi;`	`set of int: ROWS = 1..n;`
1D array of variables	`array[INDEX] of var D: name;`	`array[1..n] of var 1..n: q;`
2D array	`array[I, J] of var D: name;`	`array[1..9, 1..9] of var 1..9: grid;`
Array of parameters	`array[INDEX] of int: name;`	`array[1..3] of int: w = [2, 5, 4];`
Enumerated type	`enum NAME = {a, b, c};`	`enum COLOR = {red, green, blue};`

Tips

- Always give variables the smallest domain you can: `var 0..9` is much faster than `var int`.
- Index sets are usually named ranges (`ROWS`, `ITEMS`) so the model stays readable.
- An enum can be used as a domain: `array[REGIONS] of var COLOR: c;`

3. Constraints and operators

Each constraint starts with the keyword `constraint` and is a Boolean expression the solution must make true.

Category	Operators / keywords
Arithmetic	`+ - * div mod abs(x) pow(x, k)`
Comparison	`= != < <= > >=`
Logical	`/\` (and), `\/` (or), `not`, `->` (implies), `<->` (equivalent), `xor`
Aggregation	`sum(...)`, `product(...)`, `min(...)`, `max(...)`
Quantifiers	`forall(...)`, `exists(...)`
Set membership	`x in {1, 3, 5}`, `x in 1..5`

Generators and comprehensions — repeat a constraint over many indices:

```
% every row sums to 10
constraint forall(i in 1..n)( sum(j in 1..n)(grid[i, j]) = 10 );

% all pairs i < j are different
constraint forall(i, j in 1..n where i < j)( x[i] != x[j] );

% at least one item is chosen
constraint exists(i in ITEMS)( take[i] );
```

Conditionals

```
constraint if x > 5 then y = 1 else y = 0 endif;
```

Reification — a Boolean used as 0/1 lets you count constraints that hold:

```
var int: nb_big = sum(i in 1..n)( bool2int(x[i] > 5) );
```

Defining a variable with a value (no separate constraint needed):

```
var int: total = sum(i in ITEMS)(w[i] * take[i]);
```

4. Global constraints

Global constraints capture common patterns and are solved much faster than the equivalent small constraints. Add `include "globals.mzn";` at the top (or include one file, e.g. `include "alldifferent.mzn";`).

Constraint	Meaning	Example
`alldifferent(x)`	All values in array `x` are different	`alldifferent(q);`
`all_equal(x)`	All values are equal	`all_equal([a, b, c]);`
`count(x, v) = c`	Value `v` appears `c` times in `x`	`count(shift, 1) = 3;`
`among(c, x, S)`	Exactly `c` elements of `x` are in set `S`	`among(2, day, {6, 7});`
`element(i, a, v)` or `a[i] = v`	`v` is the `i`-th element of `a` (i is a variable)	`cost[choice] = c;`
`increasing(x)`	`x` is sorted, non-decreasing	`increasing(x);`
`table(x, T)`	Tuple `x` is one of the rows of `T`	`table([a, b], [| 1,2 | 2,3 |]);`
`cumulative(s, d, r, c)`	Tasks with start `s`, duration `d`, resource use `r` never exceed capacity `c`	`cumulative(start, dur, req, 3);`
`disjunctive(s, d)`	Tasks never overlap (one machine)	`disjunctive(start, dur);`
`circuit(x)`	`x` forms one Hamiltonian cycle (successor model)	`circuit(next);`
`bin_packing(c, bin, w)`	Items with weight `w` placed in `bin` never exceed capacity `c`	`bin_packing(10, box, w);`

Rule of thumb: if you are writing `forall(i < j)(x[i] != x[j])`, replace it with `alldifferent(x)`.

5. Solving, output and data files

Solve item — exactly one per model:

```
solve satisfy;              % find any solution
solve minimize cost;        % find the smallest cost
solve maximize profit;      % find the largest profit
```

IDE options: tick **"Print all solutions"** (or `-a` in the terminal) to list every solution of a `satisfy` problem; `--time-limit 10000` stops after 10 s.

Output — a list of strings; `\(expr)` inserts a value, `\n` is a new line:

```
output ["x = \(x)\n"];
output ["q = ", show(q), "\n"];
output [ show(grid[i, j]) ++ if j == n then "\n" else " " endif
       | i, j in 1..n ];
```

Data file (.dzn) — only assignments, no `var`, no constraints:

```
% knapsack.dzn
n = 4;
capacity = 10;
weight = [5, 4, 6, 3];
value  = [10, 40, 30, 50];
matrix = [| 1, 2, 3
          | 4, 5, 6 |];    % 2D array syntax
```

Common errors

Message or symptom	Usual cause and fix
`syntax error, unexpected ...`	A missing `;` at the end of the previous line
`undefined identifier`	Typo in a name, or a missing `include` for a global constraint
`type error: ... var int vs int`	Using a variable where a parameter is required (e.g. an array index range)
`=====UNSATISFIABLE=====`	The constraints contradict each other; remove them one at a time to find which
Solver runs forever	Domains too large (`var int`); give bounds, use global constraints
`----------`	Separator printed after each solution (normal)
`==========`	The search is complete: all solutions found or optimum proven (normal)